PROJECT WIKI

Yumeniwa

An otaku media tracker, designed and built from scratch

Chaitanya Shinde
Built 31 October 2026

CONTENTS

1 The problem 3
1.1 Problem statement 3
1.2 Goal in one sentence 3
1.3 Problems and how Yumeniwa answers them 3
1.4 Stance, status and future scope 4
2 Features 5
2.1 Guaranteed planned features for initial V1 rollout 5
2.2 Planned for versions which will come out soon after base version 5
3 Entities and relationships 6
4 Main flows 7
5 API surface 8
6 Non-functional needs 9
7 Glossary 10
7.1 Domain terms 10
7.2 Technical terms 10

CHAPTER 1

The problem

yumeniwa : one single stop leisure tracker for otakus

1.1 Problem statement

Tracking media items with help of other providers is often unreliable and has a scattered nature when various media items are to be tracked

1.2 Goal in one sentence

Let user be in control of what they want and how they want , for otakus let them track all of their likes in one single place rather than letting it spread over multiple apps , offline only static notes etc

1.3 Problems and how Yumeniwa answers them

Dependence on one provider hurts reliability

Many of the tracking apps made for otaku face one problem :

1. they are dependent on one singleservice provider , like anilist or myanimelist
   - (which can face outages at any given time for example 21 October 2026 (most recent): AniList deliberately disabled its public API with HTTP 403 (“temporarily disabled due to severe stability issues”) in response to a major PHP security vulnerability) ,
   - our application solves this by allowing user to integrate multiple surfaces to mitigate this risk such as
     1. google drive connector
     2. persistent offline backups done frequently these form something called as **Client store** , so that data is always readily stored securely in case even if one of service goes down , (for efficient backup we try diff based backups) , each Client store belongs to **clients** the user is logged in to which sync to a global **Central store**

No provider, no problem

1. Add and track as you please , even the entries which you add yourself manually

> NOTE
>
> These aren’t synced with providers, but store-based syncing is still allowed.

these are reffered to as **manual** records , treated differently from **provider** records , you own them they still stay synced across your clients

Cannot track various media and niches

1. one provider , say anilist can only offer services to track information about anime , manga at best , but what about otakus who like visual novels , gacha games as well , our solution
   - **aggregation** of providers for fetching across various media types

> NOTE
>
> V1 is restricted to a tested set of providers only.

With **bring your own providers(added very soon once base functionality works , important core aspect we wish to present as well)**

- **Only 5 media types initially**
  1. Anime
  2. Manga
  3. Light novel
  4. Visual novel
  5. Game

Bidirectional focused

(Synced across providers only if they allow write access through their API.)

1. ensures data is consistent across client and central stores , google cloud drive , providers etc seamlessly , majority of providers we offer do allow write access we will explicitly disclose which one's dont though

1.4 Stance, status and future scope

Above are the main problems faced by me the developer atleast and what I wish to solve , but beyond that I believe further improvmenets and future scopes I would love to introduce are :

- **add future socialization features:** enabling users to create and engage in social forums and such , let users write extremely rich feedbacks , create dedicated forum pages , discussion groups and so on .
- **extremely robust user taste analyzer:** detailed analytics and building user profile about their tastes , adding generative ai features , so that an in depth detailed user profile and analysis is available for further recommendations .
- **Possible media viewer(limited to anime , manga , lightnovels initially):** application shipped with media viewers for anime manga lightnovels user can add their desired content providers of actual media and engage with media and track it at same time

We hope you enjoy it once it's live

CHAPTER 2

Features

2.1 Guaranteed planned features for initial V1 rollout

> NOTE
>
> Considered from the perspective of version V1.

Must have

1. Proper Efficient Syncing mechanism across **Central** store and **Client** store
2. Should work for user who wishes to track manually with no providers
3. Functional offline (implemented by say queueing system) , for cases such as weak network , or no network at all , sync when connectivity is restored
4. persistent version based local copies maintained (solves key issue of **reliability** we wish to address)
5. Tracks atleast 5 fixed media types such as
    1. Anime
    2. Manga
    3. Light novel
    4. Visual novel
    5. Game
6. Shipped with essential providers More Research is needed to claim and confirm exact providers , ideally each provider should be
    - Exhaustive : covers enough width of media types they advertise
    - Allow api / easy auth : So that integration with our application is possible and easy

(Below is the obvious requirement but we state it anyway as)

1. Bidirectional Sync : ensure not only stores are synced but the data is obviously synced with providers themselves

Nice to have

1. **BYO(Bring Your Own Providers)** : Gives user the ability to bring their own provider / tracking service and plug it in our application for proper syncing

2.2 Planned for versions which will come out soon after base version

1. Google drive connector support for dedicated personal cloud backups
2. social forums , groups , communities
3. User Taste analyzer
4. media viewers

CHAPTER 3

Entities and relationships

> NOTE
>
> TBD

CHAPTER 4

Main flows

> NOTE
>
> TBD

CHAPTER 5

API surface

> NOTE
>
> TBD

CHAPTER 6

Non-functional needs

> NOTE
>
> TBD

CHAPTER 7

Glossary

7.1 Domain terms

Provider A third-party service that holds media data and, usually, users' lists, such as AniList, MyAnimeList or VNDB. Yumeniwa reads from it and, where its API allows, writes back. The code that talks to a provider is an **adapter** (see Technical terms).

BYO Bring your own provider: the user connects a provider that Yumeniwa does not ship with.

Client A running instance of Yumeniwa on one of the user's devices: the PWA on a phone or the web app on a PC. Each client has its own client store.

Store A database that holds a user's records. It has two specialisations:

- **Client store:** the store kept on a client's device, so that client keeps working offline.
- **Central store:** the single store treated as the source of truth; client stores sync with it. In the current draft, syncing with providers also passes through it.

Record One tracked entry (one media item on the user's list) held in a store. By origin it is one of two specialisations:

- **Manual record:** created by the user without any provider. It syncs between stores but never with a provider.
- **Provider record:** originated from a provider. It syncs between stores and, where that provider's API allows writes, with the provider.

Sync The process that brings stores to the same state: the central store's version of each record is treated as true, and client stores are updated to match it.

Conflict A state during sync in which two or more stores hold different versions of the same record. It is resolved by confirming which version is true, by comparing the central and client versions.

> **OPEN QUESTION** Still to settle
>
> - **Aggregator** is not defined yet.
> - Does the central store sync with providers, or does each client? The Store entry and my stated architecture (clients talk to providers) disagree.
> - Sync: is the central version **always** true, or only usually?
> - Conflict: if a client's version is newer and the other side has not changed since the last sync, is that a conflict or just an update? (See **three-way merge**.)

7.2 Technical terms

> **NOTE**
>
> Technical terms below were written with research assistance from Claude; key definitions link to their source. Domain terms (above) are my own.

Web and APIs

API An interface through which one program requests data or actions from another, such as AniList's interface for reading and updating lists.

Endpoint One address an API exposes, identified by a URL and an HTTP method.

HTTP The request–response protocol of the web: a client sends a request (method, URL, headers, body) and a server returns a response with a status code (MDN).

REST An architectural style for HTTP APIs in which resources have URLs and are read or changed with standard methods (GET, POST, PUT, PATCH, DELETE), and each request carries everything the server needs.

GraphQL A query language for APIs in which the client names exactly the fields it wants, usually through a single endpoint; AniList's API uses it (graphql.org).

JSON A text format for structured data built from objects, arrays, strings, numbers, booleans and null (RFC 8259).

CRUD Create, read, update, delete: the four basic operations on stored records.

Pagination Splitting a large result into pages or chunks fetched one request at a time; AniList returns a user's list in chunks of up to 500 entries.

Rate limit The maximum number of requests a client may send to an API in a time window; going over it usually returns HTTP 429 Too Many Requests.

Origin The scheme, host and port of a URL; `http://localhost:5173` and `http://localhost:8080` are different origins.

CORS Cross-Origin Resource Sharing: response headers with which a server allows pages from other origins to read its responses; the browser enforces it (MDN).

Authentication and security

OAuth 2.0 A framework that lets a user grant an application limited access to their account on another service without sharing their password (RFC 6749).

PKCE Proof Key for Code Exchange, an OAuth extension that stops a stolen authorisation code from being used; required for apps that cannot keep a secret, such as browser and mobile apps (RFC 7636).

Access token A short-lived credential sent with each request to prove what the caller may access.

Refresh token A longer-lived credential used only to obtain new access tokens when they expire.

JWT JSON Web Token, a compact, signed token that carries claims (such as a user id and an expiry time) the server can verify without a database lookup (RFC 7519).

SSRF Server-side request forgery: tricking a server into sending requests to addresses the attacker chooses, often internal ones (OWASP).

Sandbox An isolated environment that limits what running code can access, used to run code that is not trusted.

seccomp A Linux kernel feature that restricts which system calls a process may make (Cloudflare).

Clients and offline work

PWA Progressive web app: "an app that's built using web platform technologies, but that provides a user experience like that of a platform-specific app"; it can be installed and can work offline (MDN).

Web app manifest A JSON file that gives a PWA its name, icons and display mode, so the browser can install it like an app.

Service worker A script the browser runs separately from the page; it sits between the app and the network, so it can serve cached files offline and retry requests later.

WebAPK The Android package Chrome generates when a user installs a PWA, so it appears in the app drawer like a native app (web.dev).

IndexedDB A database built into the browser for storing large amounts of structured data on the device (MDN).

Offline-first A design in which the app reads and writes local data first and synchronises with servers when a connection is available.

Local-first software Software in which the user's own device holds the primary copy of their data, with sync and collaboration added on top (Ink & Switch).

Queue (outbox) A durable list of pending changes, kept so they can be sent later, for example after the network returns.

Idempotent An operation is idempotent when performing it twice has the same effect as performing it once; retries depend on this.

Diff (delta) The set of changes between two versions of some data, as opposed to a full copy.

Snapshot A complete copy of data as it stood at one moment.

Source of truth The copy whose data is treated as correct when copies disagree.

Version control (the sync model's inspiration)

Commit In Git, a snapshot of a project together with a link to its parent commit(s), named by the hash of its content.

Merge base The most recent commit two diverged histories share.

Fast-forward Updating a branch to a newer commit that already contains all of its history, so no merge is needed.

Three-way merge Combining two diverged versions using their merge base: a change on only one side is taken; changes to the same part on both sides are a conflict (Pro Git).

Design and data

Adapter A component that translates between two incompatible interfaces, such as a provider's API and Yumeniwa's own data model.

Contract A precise specification of what each side of an interface provides and expects: inputs, outputs, errors and limits.

ER model Entity–relationship model: a diagram of the things a system stores data about (entities), their attributes and the relationships between them.

Specialisation In ER modelling, splitting a general entity into subtypes that add their own attributes (its reverse is generalisation).

Stack

MongoDB A document database that stores records as JSON-like documents (BSON) in collections.

Mongoose An object-document mapper (ODM) for MongoDB in Node.js: it adds schemas, validation and model methods.

Express A minimal web framework for Node.js that routes HTTP requests through middleware to handlers.

React A JavaScript library for building user interfaces from components.

Node.js A JavaScript runtime built on the V8 engine that runs JavaScript outside the browser.

MERN MongoDB, Express, React and Node.js used together.